

APOSTILA REDESENHADA PARA APRENDER
MELHOR

Git com Python

Guia visual de memorizacao

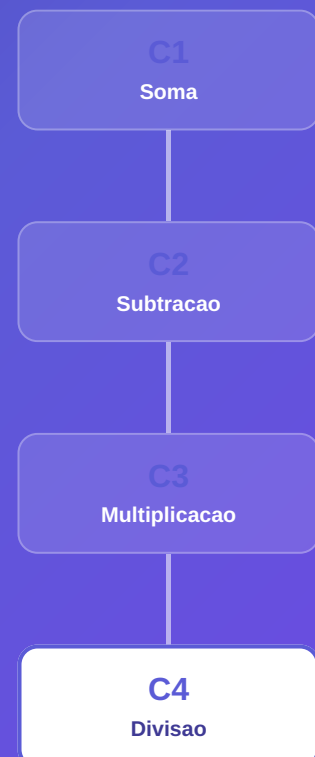
Da intuicao de controle de versao ao fluxo
status -> add -> commit -> push, com
exemplos progressivos, mapas mentais,
analogias e pratica de recuperacao.

Exemplos trabalhados

Mapas visuais

Teste de memoria

Ideia central: Git nao e uma linguagem de programacao. E um sistema de controle de versao que registra estados do projeto ao longo do tempo.



Mapa-mestre

Antes dos detalhes, memorize a historia inteira. Os comandos ficam mais facéis quando cada um responde a uma pergunta.



Frase de memoria

INICIO → VEJO → PREPARO →
REGISTRO → ENVIO → TRAGO

O ciclo que mais se repete

```
git status  
git add .  
git commit -m "o que eu fiz"  
git push
```

Como estudar: primeiro tente lembrar a funcao de cada comando sem olhar. Depois confira. Esse pequeno esforco de recuperacao fortalece a memoria.

Por que Git existe?

Sem Git

`calculadora_v1.py``calculadora_v2.py``calculadora_final.py``calculadora_final_agora_vai.py`

VS

Com Git

`calculadora.py``C1` → `C2` → `C3` → `C4`

um arquivo atual + historico recuperavel

Recuperar

voltar a versoes anteriores

Comparar

ver o que mudou

Rastrear

saber quando e por quem mudou

ANALOGIA

Git = uma maquina do tempo do projeto.

Voce continua trabalhando no arquivo atual, enquanto o repositorio guarda checkpoints do passado.

Na apostila, essa necessidade nasce do controle de versao manual: criar copias sucessivas funciona, mas se torna dificil de manter e colaborar.

git init

Cenário

Voce criou uma pasta `Calculadora/` e abriu no VS Code. O Git ainda nao acompanha nada.

ANTES

```
Calculadora/  
calculadora.py
```

pasta comum

→ `git init` →

DEPOIS

```
Calculadora/  
calculadora.py  
.git/
```

repositorio Git local

O que muda?

O Git cria a pasta oculta `.git`, onde ficam metadados e objetos do repositorio.

Gancho de memoria

INIT = INICIAR

Normalmente voce executa uma vez por projeto criado do zero.

Pergunta que ativa o comando: "Quero transformar esta pasta em um repositorio Git. O que uso?"

`git init`

Estados dos arquivos

Esta é a parte mais importante para entender `status`, `add` e `commit` sem decorar mecanicamente.



Modelo mental: EDITAR → **MODIFIED** → `git add` → **STAGED** → `git commit` → **UNMODIFIED**

Tracked

Arquivo que o Git conhece. Pode estar **unmodified**, **modified** ou **staged**.

Untracked

Arquivo presente no diretório, mas que ainda não faz parte do histórico controlado pelo Git.

Regra de ouro: o commit registra o que está em **Staged**. Pensar nisso evita a maioria das confusões iniciais.

git status

Voce criou `calculadora.py`

Agora quer descobrir como o Git enxerga o projeto.

`git status`

"Git, o que esta acontecendo agora?"

Terminal - exemplo simplificado

```
$ git status
Untracked files:
  calculadora.py
```

Versao completa

`git status`

Mostra branch e situacao detalhada dos arquivos.

Versao curta

`git status -s`

Mostra os estados de forma compacta.

ANALOGIA

status = painel do carro.

Antes de agir, voce olha o painel para saber a situacao.

Atalho mental: STATUS = **VERIFICAR**, nao alterar.

git add

O status mostrou um arquivo novo

Voce quer que ele participe do proximo registro.

Alteracoes

calculadora.py

teste.py

README.md

git add calculadora.py



STAGING AREA

calculadora.py

conteudo selecionado para o proximo commit

O que `add` significa?

"Inclua exatamente este conteudo no proximo commit."

E `git add .` ?

Prepara as alteracoes dentro do diretorio atual (e subdiretorios, conforme o caso).

ADD escolhe. COMMIT registra.

Pergunta que ativa o comando: "Quais mudancas quero colocar no proximo checkpoint?"

`git add`

git commit

A staging area ja esta pronta

Agora voce quer transformar aquela selecao em um ponto do historico.

```
git commit -m "funcao somar implementada"
```

"Registre este estado no historico."

STAGED

calculadora.py

→ commit →

C1

funcao somar implementada

snapshot registrado

C1

Soma

C2

Subtracao

C3

Multiplicacao

C4

Divisao

ANALOGIA

Commit = checkpoint de jogo.

Voce pode continuar jogando sem perder aquele ponto registrado.

Detalhe importante: arquivos que ainda nao foram preparados com `git add` nao entram nesse commit (salvo atalhos especificos, como `commit -a` para arquivos ja rastreados).

Exemplo prático progressivo

Agora conecte os comandos a uma historia concreta. A calculadora cresce e o Git registra cada passo.

1

Crie a soma

```
def somar(a, b):  
    return a + b
```

```
git status → git add calculadora.py → git commit -m "soma"
```

2

Adicione o teste

```
import calculadora  
print(calculadora.somar(1, 2))
```

```
status → add teste_calculadora.py → commit
```

3

Implemente subtracao

```
def subtrair(a, b):  
    return a - b
```

```
status → add → commit
```

Padrao que se repete: MUDAR → OLHAR → PREPARAR → REGISTRAR

Autoexplicacao: por que a apostila separa implementacao e teste em commits diferentes? Porque o `git add` permite selecionar exatamente o que entra em cada snapshot.

Historico e posição atual

git log

Passado: mostra os commits feitos.

"Como cheguei ate aqui?"

git diff

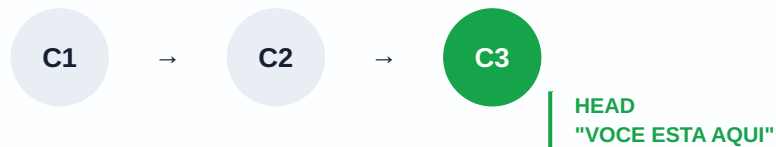
Diferença: mostra linhas alteradas que ainda nao foram preparadas (por padrao).

"O que mudou exatamente?"

HEAD

Posicao atual: referencia o ponto em que voce esta trabalhando.

"Onde estou?"



Historico compacto

```
$ git log --pretty=oneline
9a2... divisao
61f... multiplicacao
2bd... subtracao
0c1... soma
```

Trinca de memoria

STATUS = presente

LOG = passado

HEAD = posicao atual

Branches

Uma branch permite que o desenvolvimento siga outro caminho a partir de um commit.

main

A



B



C



D

feature

C



E



F

Commit

fotografia do estado

Branch

estrada de desenvolvimento

HEAD

"voce esta aqui"

ANALOGIA VISUAL

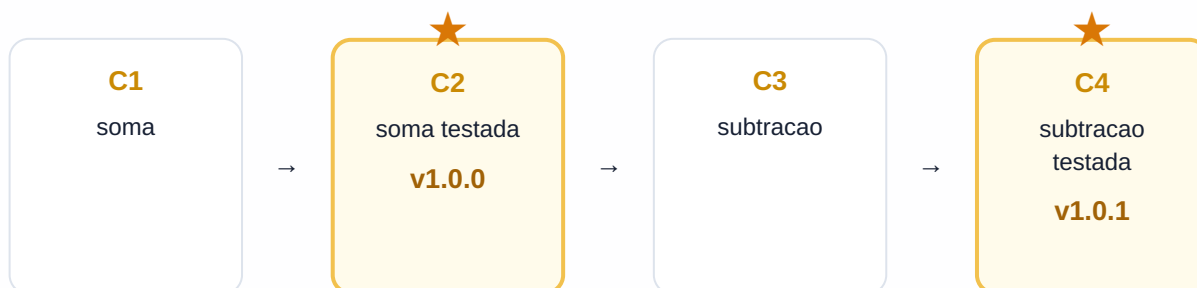
Commit = foto | Branch = estrada | HEAD = marcador de localizacao

Esse trio ajuda a entender por que o mesmo repositorio pode ter varios rumos de desenvolvimento.

Imagine: voce quer testar uma nova interface sem mexer imediatamente na linha principal. Conceitualmente, uma branch cria esse caminho paralelo.

Tags

Nem todo commit precisa receber um nome especial. Tags servem para destacar marcos importantes.



```
git tag -a v1.0.0 -m "soma implementada e testada"
```

"Marque este commit como um marco importante."

Ver tags

```
git tag  
git tag --list
```

Inspecionar tag

```
git show v1.0.0
```

Commit = registrar. Tag = destacar um registro.

Remote, origin, push e pull

Ate aqui, tudo pode existir apenas no seu computador. Um remote conecta seu repositório local a outro repositório.



origin

Nome convencional usado como apelido para um repositório remoto.

```
git remote add origin URL
```

Direcao

PUSH = local → remoto

PULL = remoto → local

```
git push origin main
```

envia a branch **main** para o remote chamado **origin**

Atualizacao pratica de seguranca: nao coloque tokens de acesso em apostilas, URLs compartilhadas ou commits. Prefira mecanismos seguros de credenciais (por exemplo, Git Credential Manager) ou SSH.

Os 6 comandos em uma história

Uma unica historia com os 6 comandos. Leia de cima para baixo e tente prever o proximo passo antes de olhar.

1

Criei a pasta e o arquivo.

Quero iniciar o controle de versao.

`git init`

2

Criei `calculadora.py`.

Quero saber o que o Git esta vendo.

`git status`

3

Quero incluir a mudanca no proximo registro.

Preparo o conteudo.

`git add .`

4

O estado esta pronto.

Registro o checkpoint.

`git commit -m "primeira versao"`

5

Quero compartilhar/backup remoto.

Envio os commits.

`git push`

6

O remoto recebeu mudancas feitas em outro lugar.

Trago essas mudancas.

`git pull`**INIT → STATUS → ADD → COMMIT → PUSH ↔ PULL**

Teste de memória

Responda sem voltar as paginas anteriores. O objetivo aqui e **puxar a informacao da memoria**, nao reconhecer a resposta.

1 Começar

Transformar uma pasta comum em repositório Git.

`git _____`

2 Verificar

Descobrir quais arquivos estão modificados ou não rastreados.

`git _____`

3 Preparar

Escolher o conteúdo que entrará no próximo checkpoint.

`git _____`

4 Registrar

Registrar um snapshot no histórico.

`git _____`

5 Histórico

Visualizar a linha do tempo de commits.

`git _____`

6 Destacar

Marcar um commit importante como v1.0.0.

`git _____`

7 Enviar

Enviar commits locais para o remoto.

`git _____`

8 Trazer

Trazer mudanças do remoto para o repositório local.

`git _____`

Regra: só vire a página depois de tentar todas. Errar aqui é útil: o erro mostra exatamente o que precisa de mais recuperação.

Gabarito e plano de revisão

Gabarito

1 <code>git init</code>	2 <code>git status</code>
3 <code>git add</code>	4 <code>git commit</code>
5 <code>git log</code>	6 <code>git tag</code>
7 <code>git push</code>	8 <code>git pull</code>

Desafio

Use `git add nome_do_arquivo` apenas no arquivo desejado; depois faça o commit. A staging area permite selecionar o conteúdo do snapshot.

Revisao espacada sugerida

Hoje

reconte o fluxo sem olhar

Amanha

refaca o quiz + 1 exemplo no terminal

3 dias

crie outro mini projeto e use o ciclo

7 dias

explique Git em voz alta em 2 minutos

Desafio de transferencia: voce alterou 3 arquivos, mas quer que apenas 1 entre no proximo commit.

Resposta: use `git add nome_do_arquivo` somente no arquivo desejado e depois faca o commit.

MODELO FINAL

TRABALHAR → **STATUS** → **ADD** → **COMMIT** → **PUSH**

e use **PULL** quando precisar trazer atualizacoes remotas.

Fontes e metodologia

Conteudo-base: apostila enviada, secoes 2.1 a 2.11 (controle de versao, estados, calculadora Python, branches, tags e repositorio remoto).

Git: Chacon, S.; Straub, B. *Pro Git*, 2a ed.; documentacao oficial git-scm.com, especialmente "Getting a Git Repository", "Recording Changes to the Repository" e "What is Git?".

Aprendizagem: McDermott (2021), *Practicing Retrieval Facilitates Learning*; van Gog, Paas & Sweller (2010), *Cognitive Load Theory: Advances in Research on Worked Examples...*; Cepeda et al. (2008), *Spacing Effects in Learning*.

Este guia reorganiza o conteudo para reduzir carga cognitiva, usar exemplos trabalhados, associacoes visuais e pratica de recuperacao. Ele nao substitui a apostila original; funciona como camada de estudo e memorizacao.